



How to Set and List Environment Variables in Linux

Updated on Jun 3, 2021 • 6 min read

Environment Variables

In Linux and Unix based systems environment variables are a set of dynamic named values, stored within the system that are used by applications launched in shells or subshells. In simple words, an environment variable is a variable with a name and an associated value.

Environment variables allow you to customize how the system works and the behavior of the applications on the system. For example, the environment variable can store information about the default [text editor](#) or browser, the path to executable files, or the system locale and keyboard layout settings.

In this guide, we will explain to read and set environment and shell variables.

Environment Variables and Shell Variables

Variables have the following format:

```
KEY=value  
KEY="Some other value"  
KEY=value1:value2
```

- The names of the variables are case-sensitive. By convention, environment variables should have UPPER CASE names.
- When assigning multiple values to the variable they must be separated by the colon : character.



Variables can be classified into two main categories, environment variables, and shell variables.

Environment variables are variables that are available system-wide and are inherited by all spawned child processes and shells.

Shell variables are variables that apply only to the current shell instance. Each shell such as `zsh` and `bash`, has its own set of internal shell variables.

There are several commands available that allow you to list and set environment variables in Linux:

- `env` – The command allows you to run another program in a custom environment without modifying the current one. When used without an argument it will print a list of the current environment variables.
- `printenv` – The command prints all or the specified environment variables.
- `set` – The command sets or unsets shell variables. When used without an argument it will print a list of all variables including environment and shell variables, and shell functions.
- `unset` – The command deletes shell and environment variables.
- `export` – The command sets environment variables.

List Environment Variables

The most used command to displays the environment variables is `printenv`. If the name of the variable is passed as an argument to the command, only the value of that variable is displayed. If no argument is specified, `printenv` prints a list of all environment variables, one variable per line.

For example, to display the value of the `HOME` environment variable you would run:

```
$ printenv HOME
```

The output will print the path of the currently logged in user:

```
Output
/home/linuxize
```

You can also pass more than one arguments to the `printenv` command:

```
$ printenv LANG PWD
```

```
Output
en_US
/home/linuxize
```



```
$ printenv
```

The output will look something like this:

Output

```
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:...
LESSCLOSE=/usr/bin/lesspipe %s %s
LANG=en_US
S_COLORS=auto
XDG_SESSION_ID=5
USER=linuxize
PWD=/home/linuxize
HOME=/home/linuxize
SSH_CLIENT=192.168.121.1 34422 22
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
SSH_TTY=/dev/pts/0
MAIL=/var/mail/linuxize
TERM=xterm-256color
SHELL=/bin/bash
SHLVL=1
LANGUAGE=en_US:
LOGNAME=linuxize
XDG_RUNTIME_DIR=/run/user/1000
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/loca
LESSOPEN=| /usr/bin/lesspipe %s
_=/usr/bin/printenv
```

Below are some of the most common environment variables:

- **USER** - The current logged in user.
- **HOME** - The home directory of the current user.
- **EDITOR** - The default file editor to be used. This is the editor that will be used when you type `edit` in your terminal.
- **SHELL** - The path of the current user's shell, such as `bash` or `zsh`.
- **LOGNAME** - The name of the current user.
- **PATH** - A list of directories to be searched when executing commands. When you run a command the system will search those directories in this order and use the first found executable.
- **LANG** - The current locales settings.
- **TERM** - The current terminal emulation.
- **MAIL** - Location of where the current user's mail is stored.

The `printenv` and `env` commands print only the environment variables. If you want to get a list of



```
$ set
```

Output

```
BASH=/bin/bash
BASHOPTS=checkwinsize:cmdhist:complete_fullquote:expand_aliases:extglob:extquote:force
BASH_ALIASES=()
BASH_ARGC=()
BASH_ARGV=()
```

The command will display a large list of all variables so you probably want to pipe the output to the [less](#) command.

```
$ set | less
```

You can also use the [echo command](#) to print a shell variable. For example, to print the value of the `BASH_VERSION` variable you would run:

```
$ echo $BASH_VERSION
```

Output

```
4.4.19(1)-release
```

Setting Environment Variables

To better illustrate the difference between the Shell and Environment variables we'll start with setting Shell Variables and then move on to the Environment variables.

To create a new shell variable with the name `MY_VAR` and value `Linuxize` simply type:

```
$ MY_VAR='Linuxize'
```

You can verify that the variable is set by using either `echo $MY_VAR` or filtering the output of the `set` command with [grep](#) `set | grep MY_VAR`:

```
$ echo $MY_VAR
```

Output

```
Linuxize
```

Use the `printenv` command to check whether this variable is an environment variable or not:



The output will be empty which tell us that the variable is not an environment variable.

You can also try to print the variable in a new shell and you will get an empty output.

```
$ bash -c 'echo $MY_VAR'
```

The `export` command is used to set Environment variables.

To create an environment variable simply export the shell variable as an environment variable:

```
$ export MY_VAR
```

You can check this by running:

```
$ printenv MY_VAR
```

Output
Linuxize

If you try to print the variable in a new shell this time you will get the variable name printed on your terminal:

```
$ bash -c 'echo $MY_VAR'
```

Output
Linuxize

You can also set environment variables in a single line:

```
$ export MY_NEW_VAR="My New Var"
```

Environment Variables created in this way are available only in the current session. If you open a new shell or if you log out all variables will be lost.

Persistent Environment Variables

To make Environment variables persistent you need to define those variables in the bash configuration files. In most Linux distributions when you start a new session, environment variables are read from the following files:

- `/etc/environment` - Use this file to set up system-wide environment variables. Variables in this file are set in the following format:



```
$ VAR_TEST="test var"
```

- `/etc/profile` - Variables set in this file are loaded whenever a bash login shell is entered. When declaring environment variables in this file you need to use the `export` command:

```
$ export JAVA_HOME="/path/to/java/home"
$ export PATH=$PATH:$JAVA_HOME/bin
```

- Per-user shell specific configuration files. For example, if you are using Bash, you can declare the variables in the `~/.bashrc`:

```
$ export PATH="$HOME/bin:$PATH"
```

To load the new environment variables into the current shell session use the [source](#) command:

```
$ source ~/.bashrc
```

Conclusion

In this guide, we have shown you how to set and list environment and shell variables.

Feel free to leave a comment if you have any questions.

terminal

If you like our content, please consider buying us a coffee.
Thank you for your support!



BUY ME A COFFEE

Sign up to our newsletter and get our latest tutorials and news straight to your mailbox.



Subscribe

We'll never share your email address or spam you.

Related Articles

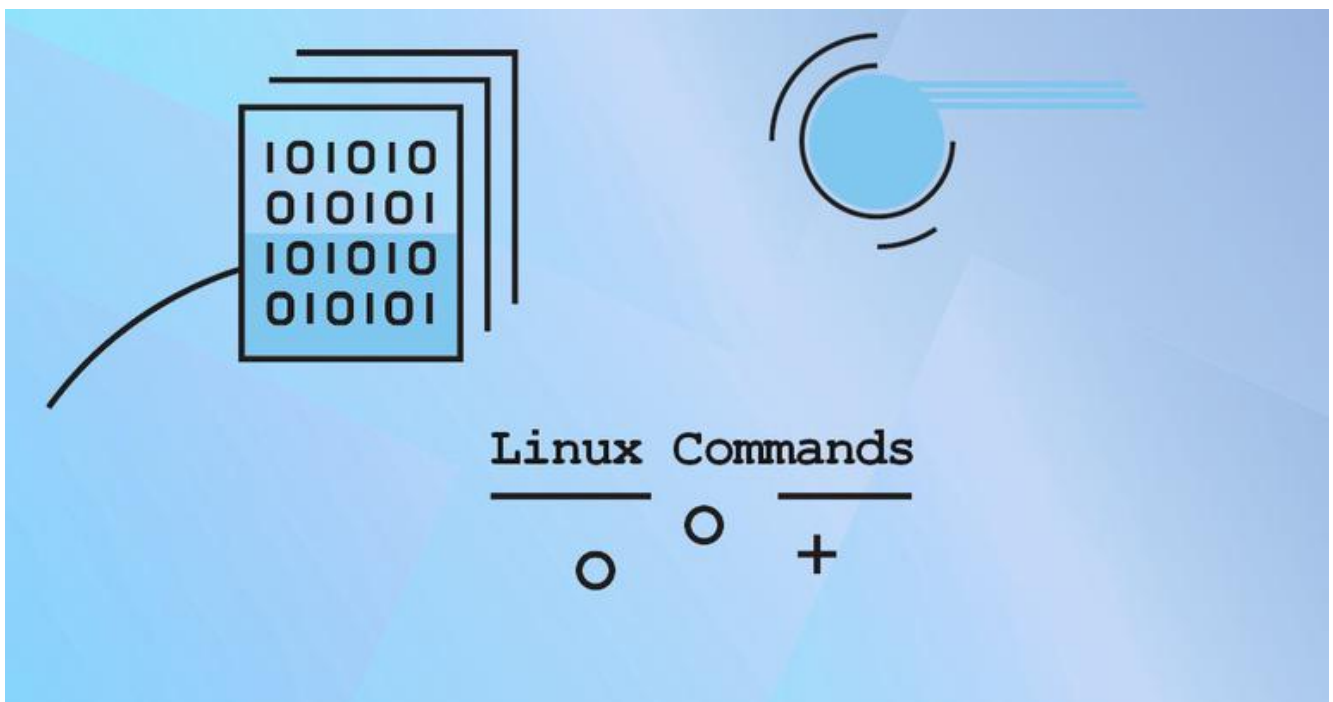
DEC 27, 2019

Understanding the /etc/shadow File



NOV 18, 2020

Basic Linux Commands





Append to File

Write a comment

Please enable JavaScript to view the comments powered by Disqus.

© 2025 Linuxize.com | A Raptive Partner Site

[Privacy Policy](#) [Terms](#) [Contact](#)

